

# Threat Detection Using Wazuh and MITRE ATT&CK

Project proposal: context analysis, scope, and prototype technical stack  
7 September 2026

## 1. Context Analysis

### My understanding of the project

My task is to implement threat detection with Wazuh across a Windows and a Linux machine, and to use MITRE ATT&CK to select the behaviours to detect and explain the resulting alerts.

### Proposed environment

I propose a controlled lab with one Windows machine, one Linux machine, and a central Wazuh deployment. My initial focus is login activity, local account creation, and scheduled tasks or jobs.

I chose these areas because they cover attempts to access accounts and behaviours that can help maintain access or recurring execution. Each has a Windows and Linux equivalent that I can demonstrate using built-in system functions.

### Role of Wazuh

I plan to use Wazuh to collect security events from both machines, analyse them centrally, and review the resulting alerts. It is an open-source platform with SIEM and XDR capabilities. [This is the Wazuh overview as per the Getting Started documentation.](#)

The main components I will be working with are:

| Component        | Responsibility                                                                         |
|------------------|----------------------------------------------------------------------------------------|
| Agent            | Runs on each monitored machine and collects the configured logs and events.            |
| Server / manager | Extracts fields from events, checks detection rules, and generates alerts.             |
| Indexer          | Stores alerts so they can be searched and retrieved.                                   |
| Dashboard        | Provides the interface for reviewing alerts, supporting evidence, and ATT&CK mappings. |

### Role of MITRE ATT&CK

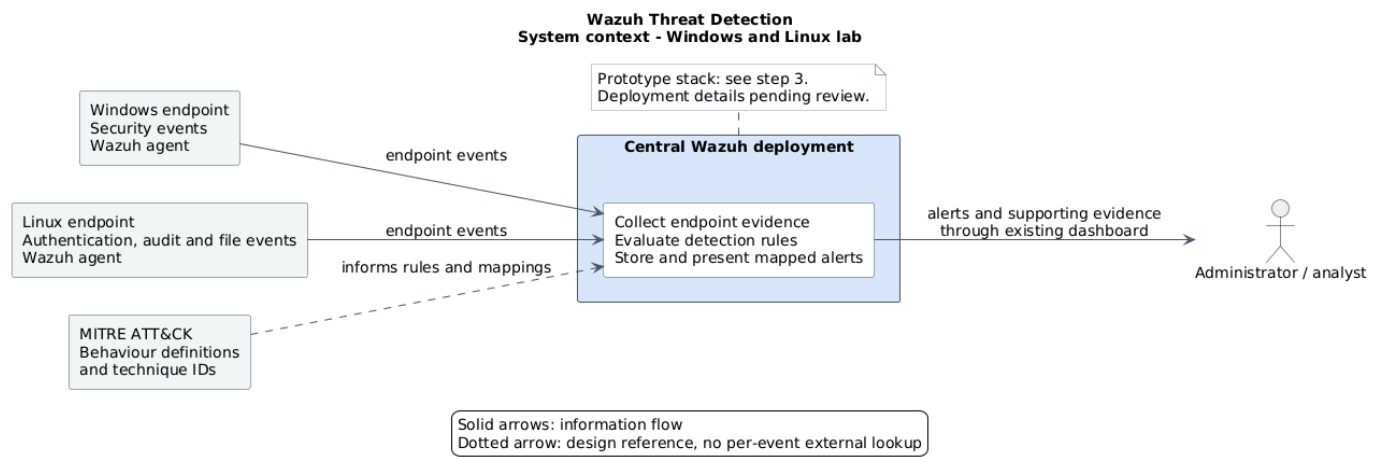
MITRE ATT&CK connects detections to documented attacker behaviour. A tactic describes an attacker's objective; techniques and sub-techniques describe the methods used. I will use these terms to explain what each detection looks for.

Wazuh ships with ATT&CK mappings on its built-in rules, and a custom rule declares its technique with a `<mitre>` block containing the technique ID. I will check the built-in rules against my scenarios first and only write custom rules where there is an actual gap.

### What this means for the implementation

- I need to verify event collection and rule behaviour before claiming a detection works.
- Windows and Linux need separate tests because their event sources and some technique mappings differ.
- Account creation and scheduled jobs can be legitimate. Alerts need context for an analyst to assess them.
- I will limit coverage claims to the procedures and conditions I have tested.

## System context



Detailed architecture will follow research and review of the prototype stack.

## Approach

I will focus on backend configuration and detection logic, with concise documentation and script output.

## 2. Problem Statement and Scope

### Problem statement

The problem I want to address is how to turn Windows and Linux security events into useful, testable detections. Login attempts, account changes, and scheduled jobs appear in different logs, which can make suspicious activity difficult to recognise and investigate.

Following the three areas identified in step 1, I propose detecting repeated password-login failures, local account creation, and scheduled execution that could support persistence. I will use Wazuh to collect the events, map relevant alerts to MITRE ATT&CK, and retain supporting evidence. Controlled tests and legitimate comparisons will establish the demonstrated capabilities and limitations.

### Objective

My objective is a repeatable demonstration of three scenarios on both operating systems, supported by documented rules, ATT&CK mappings, and searchable alerts.

### Scope boundaries

I propose keeping the first version within the following boundaries:

| Included                                                                  | Excluded from the initial release                                         |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------|
| One Windows endpoint, one Linux endpoint, and a central Wazuh deployment. | Production rollout, high availability, and enterprise-scale testing.      |
| Required log collection and selected audit/file monitoring.               | Broad network, cloud, or Active Directory monitoring.                     |
| Existing rules, justified custom rules, and ATT&CK mappings.              | Full ATT&CK coverage, machine learning, and malware analysis.             |
| Existing dashboard and evidence exports.                                  | Custom frontend, separate application API, or replacement alert database. |
| Controlled demonstrations and benign comparison tests.                    | Automated blocking, isolation, and remediation.                           |

### Detection scenarios

Each scenario will have a Windows and Linux test, giving six cases. I will verify the required rules, audit settings, and event fields before implementation.

Event 4698 is not logged by default. It requires the "Audit Other Object Access Events" audit subcategory to be enabled, which I will configure as part of the Windows setup.

| ID | Behaviour and expected result                                                                   | Windows evidence                                                          | Linux evidence                                                                                                                   | ATT&CK mapping                                                                                |
|----|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| S1 | Alert on repeated failed logons against one account within a defined threshold and time window. | Event ID 4625, using logon type, failure status codes and source address. | SSH failures in <code>/var/log/auth.log</code> , with account and source IP.                                                     | <a href="#">T1110.001: Password Guessing</a>                                                  |
| S2 | Identify successful local account creation and the initiating user where available.             | Event ID 4720.                                                            | <code>useradd</code> records in <code>/var/log/auth.log</code> , plus <code>auditd</code> context.                               | <a href="#">T1136.001: Local Account</a>                                                      |
| S3 | Identify a new scheduled task or cron job for persistence review.                               | Event ID 4698, including the task definition XML.                         | Changes to <code>/etc/crontab</code> , <code>/etc/cron.*</code> and user crontabs, via file monitoring and <code>auditd</code> . | Windows: <a href="#">T1053.005: Scheduled Task</a> . Linux: <a href="#">T1053.003: Cron</a> . |

S1 addresses Credential Access; S2 and S3 focus on Persistence. Other uses of the scheduling techniques are outside these demonstrations.

For S2 and S3, I will report the observed action and context for review. These actions can be legitimate, and a generic file-change alert alone would not demonstrate either behaviour.

## Requirements and acceptance criteria

These are the checks I propose using to judge whether the first version is complete:

| ID | Requirement                                            | Evidence of completion                                                                                                               |
|----|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| R1 | Collect the required evidence from both endpoints.     | Received events contain the fields needed for S1 to S3. Missing telemetry is recorded as a gap.                                      |
| R2 | Detect each behaviour with a justified ATT&CK mapping. | All six cases produce the expected rule result and mapping in two consecutive controlled runs.                                       |
| R3 | Provide useful alert context.                          | Endpoint, timestamp, rule, severity, technique ID, and relevant account, source, task, or file details.                              |
| R4 | Evaluate legitimate activity and limitations.          | Compare a mistyped password, approved account creation, and an approved scheduled job on each OS against the expected results below. |
| R5 | Make the demonstration reproducible.                   | Versioned configuration and rules, setup notes, and test records linking S1 to S3 with expected results and evidence.                |

For R4, a single mistyped password below the S1 threshold should not trigger the repeated-failure alert. Approved account creation and scheduled jobs may correctly trigger S2 and S3 observation alerts. I will record them as legitimate activity; an alert alone will not be treated as proof of an attack.

I will test rules with sample events, then verify the complete path from live endpoint activity to searchable alerts.

I will measure alert delay and agree a timing target after establishing the lab baseline. Results will include missed cases and unwanted alerts, with conclusions limited to the tested conditions.

## Completion boundary and next step

I will consider the first version complete when R1 to R5 are demonstrated and limitations are documented. These requirements are proposed for review.

### 3. Prototype Technical Stack and Approach

This stack is not confirmed for implementation. It needs further research, feasibility checks, and mentor review. I have not deployed or tested the prototype yet.

#### Proposed direction

To support the six cases and acceptance criteria in step 2, I propose a virtual lab with a central Wazuh deployment and separate Windows and Linux endpoints. I would use Wazuh's existing components and small scripts to make setup and testing repeatable.

The main alert path would be:

```
Windows / Linux -> Agents -> Manager -> Filebeat -> Indexer
```

The manager, Filebeat, indexer, and dashboard would run together on the central Linux VM. Filebeat forwards generated alerts to the indexer, and the dashboard queries them for display.

I am targeting the 4.14.x branch, which is the current stable release. Wazuh 5.0 is still in beta and changes this path significantly: it removes Filebeat in favour of a built-in indexer connector and moves analysis out of the manager into the indexer. I am staying on 4.x for this project rather than building against a moving target.

#### Candidate stack

| Area              | Initial choice                                                                                        | Reason for considering it                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Lab environment   | VirtualBox, three VMs, and a private lab network.                                                     | Separate the platform and endpoints, with snapshots for endpoint resets. Check host compatibility and capacity. |
| Central platform  | Ubuntu Server 24.04 LTS and Wazuh 4.14.x release, using an all-in-one installation.                   | Keep the manager, indexer, and dashboard together.                                                              |
| Windows endpoint  | Windows 11, Wazuh agent, and selected Windows Security events.                                        | Collect authentication, account-creation, and scheduled-task evidence with appropriate audit policies.          |
| Linux endpoint    | Ubuntu Server 24.04 LTS, Wazuh agent, authentication logs, Linux Audit, and targeted file monitoring. | Collect login, account-management, and cron evidence. Verify actual fields before writing rules.                |
| Setup and testing | PowerShell, Bash, and Wazuh's rule-testing facility.                                                  | Make the Windows and Linux procedures repeatable.                                                               |
| Project records   | Git, Markdown, PlantUML, and exported alert JSON; Python as an optional reporting helper.             | Keep configuration, decisions, diagrams, and evidence together.                                                 |

Wazuh's Quickstart recommends 4 vCPU, 8 GiB RAM, and 50 GB storage for 1 to 25 agents and a 90-day alert-storage estimate. This is a reference for the central VM; endpoints, snapshots, and the host OS need additional resources. Retention still needs review. [This is the Wazuh Quickstart guidance on installation and resource requirements.](#)

## Detection logic

Detection for S1 uses Wazuh's frequency options, and the ATT&CK mapping is declared on the rule itself:

```
<rule id="100001" level="10" frequency="6" timeframe="120">
  <if_matched_sid>5716</if_matched_sid>
  <description>Repeated SSH authentication failures for the same account</description>
  <mitre>
    <id>T1110.001</id>
  </mitre>
</rule>
```

Rule IDs, thresholds and the parent SID need confirming against my actual install.

## Working methods

I would combine three established approaches:

| Approach                              | How I would apply it                                                                                                        |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Iterative prototyping                 | Complete one scenario end to end, review it, and extend the setup to the remaining cases.                                   |
| Threat-informed detection engineering | Start with an ATT&CK behaviour, identify observable events, define a detection, and test simulated and legitimate activity. |
| Detection as code                     | Keep rules, settings, sample events, and expected results in Git. Review changes and rerun affected cases.                  |

I would use short simulations of selected behaviours, drawing on [MITRE's approach to focused emulation plans](#).

Each rule and test record would link to its scenario and acceptance criteria from step 2.

## Proposed demonstration format

For each scenario, I would show Windows and Linux side by side: the action, source event, matched rule, ATT&CK mapping, alert, and legitimate comparison. Keeping the results before tuning would make improvements from configuration or rule changes visible.

A small Python script could assemble this report from exported alert JSON, checking the endpoint, time window, and expected rule result. Report automation is optional and needs review.

I would use `wazuh-logtest` for parsing and rule checks, followed by live endpoint tests to verify alert delivery. This would help distinguish missing telemetry from a rule that failed to match. [This is how Wazuh supports testing decoders and rules](#).

## Research and review before implementation

- Confirm host capacity, Windows VM requirements, and snapshot storage.
- Verify OS/Wazuh compatibility, audit settings, fields, and existing rules against actual events. Select the Windows login method and failure records for S1. Assess whether Windows needs additional process evidence from Sysmon.
- Review relevant Atomic Red Team tests, including prerequisites, actions, and cleanup. [This is the test library being considered](#).
- Agree thresholds, retention, schedule, and report format. Define lab access controls, agent enrolment, and authenticated connections.